Tema 2: Fundamentos de Programación en Python

2DAW - Valle del Miro - Curso 2025-2026



Índice

Estructura básica de programas Python

Bloques fundamentales, indentación y sintaxis básica

Variables y tipos de datos

Declaración, asignación y manipulación

Constantes y literales

Definición y usos prácticos

Operadores en Python

Aritméticos, lógicos, de comparación y bitwise

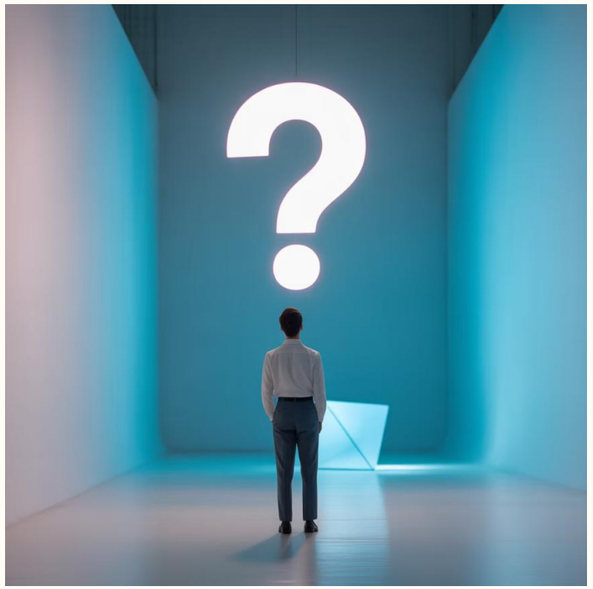
Conversión de tipos

Conversiones explícitas e implícitas

Documentación de código

Comentarios y buenas prácticas

¡Antes de empezar!



Funciones Predefinidas: `print()` e `input()`

Para poder empezar a operar con Python, vamos a ver dos funciones principales que nos permitirán visualizar información e interactuar con el código.

La función `print()`

Utilizada para mostrar mensajes, variables y otros datos en la consola. Es esencial para depurar y para la salida de información en cualquier programa.

```
nombre = "Ana"
edad = 30
print("Hola, mi nombre es", nombre)
print(f"Tengo {edad} años.")
print("La suma de 5 + 3 es:", 5 + 3)
```

La función `input()`

Permite al programa pausarse y esperar a que el usuario introduzca texto a través del teclado. El valor introducido siempre se retorna como una cadena de texto (`str`).

```
saludo = "Por favor, introduce tu nombre: "
nombre_usuario = input(saludo)
print(f"¡Bienvenido, {nombre_usuario}!")

edad_str = input("¿Cuántos años tienes? ")
edad_int = int(edad_str) # Conversión a entero
print(f"Tu edad es {edad_int}.")
```

Estructura Básica de un Programa Python

Comprendiendo los pilares fundamentales del código Python.

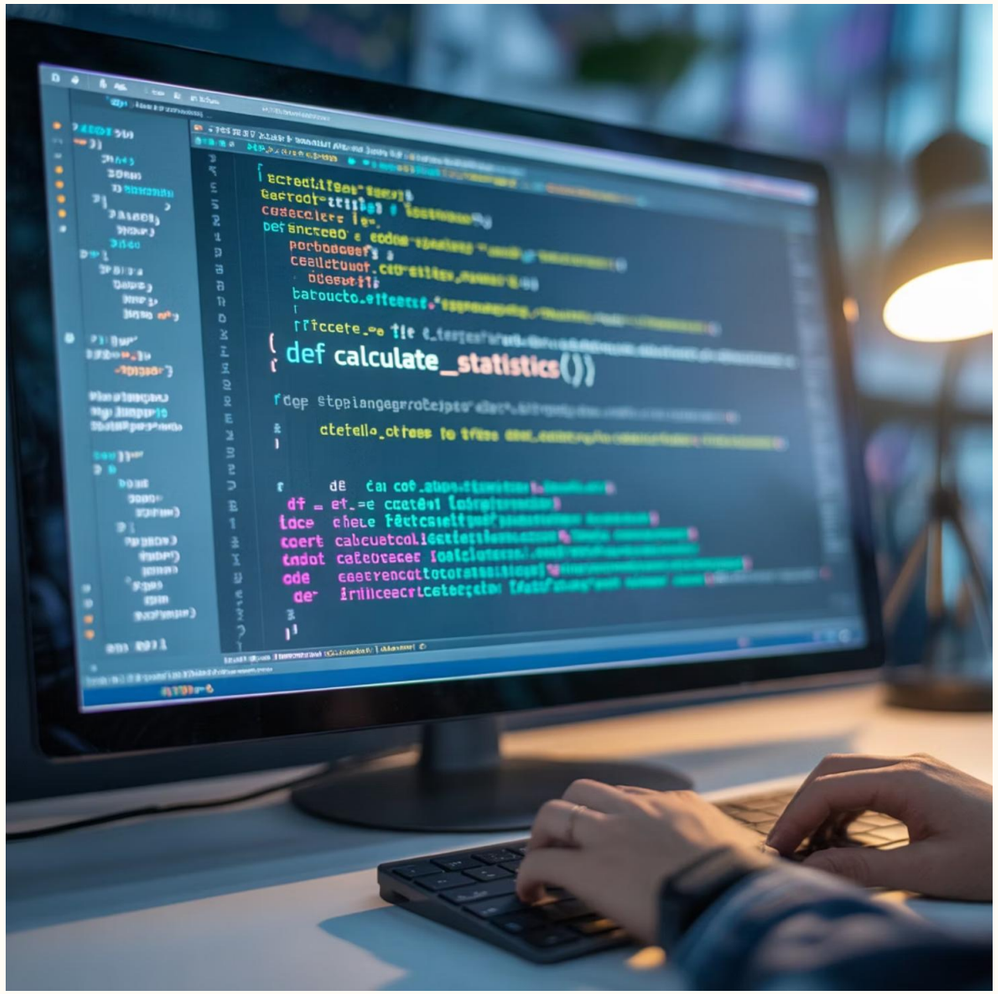


Estructura Básica de un Programa Python

Un programa Python está compuesto por diferentes bloques de código organizados de manera lógica. A diferencia de otros lenguajes, Python utiliza la indentación (espacios o tabulaciones al principio de una línea) para definir estos bloques, no llaves {} como hacen otros programas, lo que hace que el código sea naturalmente más legible.

Elementos principales:

- Módulos e importaciones: Para reutilizar código de bibliotecas
- Declaraciones y asignaciones: Donde creamos y manipulamos variables
- Funciones: Bloques de código reutilizables con un propósito específico
- Clases: Plantillas para crear objetos (programación orientada a objetos)
- Bloques condicionales: Para tomar decisiones (if, elif, else)
- Bucles: Para repetir acciones (for, while)
- Manejadores de excepciones: Para controlar errores (try, except)
- Comentarios: Para documentar el código



Ejemplo de Estructura Básica

```
# Importaciones de módulos
import math
import random

# Declaración de variables
radio = 5
altura = 10

# Función para calcular el volumen
def calcular_volumen_cilindro(r, h):
    """Calcula el volumen de un cilindro."""
    return math.pi * r**2 * h

# Condicional
if radio > 0 and altura > 0:
    volumen = calcular_volumen_cilindro(radio, altura)
    print(f"El volumen del cilindro es: {volumen}")
else:
    print("El radio y la altura deben ser positivos")

# Bucle
for i in range(3):
    valor_aleatorio = random.randint(1, 10)
    print(f"Número aleatorio {i+1}: {valor_aleatorio}")

# Manejo de excepciones
try:
    resultado = 10 / 0
except ZeroDivisionError:
    print("Error: División por cero")
```

Indentación en Python

La indentación en Python es un aspecto fundamental y diferenciador respecto a otros lenguajes de programación. No es simplemente una cuestión estética o de estilo, sino que es parte de la sintaxis del lenguaje y determina la estructura lógica del código.

Características principales:

- Obligatoria: No es opcional como en otros lenguajes
- Delimita bloques: Sustituye a las llaves {} de lenguajes como Java o C++
- Coherencia: Debe ser consistente en todo el código (espacios o tabulaciones, pero no mezclarlos)
- Nivel jerárquico: Cada nivel de indentación representa un nivel de anidamiento en la lógica del programa

La convención de estilo PEP 8 recomienda usar 4 espacios por nivel de indentación. La mayoría de editores e IDEs para Python configuran automáticamente esta convención.

Ejemplo de Errores de Indentación

Código Incorrecto

```
def calcular_promedio(numeros):
    suma = 0
    for numero in numeros:
        suma += numero
    promedio = suma / len(numeros)
        return promedio  # Error:indentación
incorrecta

valores = [10, 20, 30, 40]
resultado = calcular_promedio(valores)
print(resultado)
```

Error: La línea con `return` tiene una indentación excesiva. Debería estar al mismo nivel que la línea de cálculo del promedio, no dentro del bucle `for`.

Código Correcto

```
def calcular_promedio(numeros):
    suma = 0
    for numero in numeros:
        suma += numero
    promedio = suma / len(numeros)
    return promedio  # Correcto

valores = [10, 20, 30, 40]
resultado = calcular_promedio(valores)
print(resultado)
```

Corrección: Ahora el `return` está al nivel adecuado, indicando que se ejecuta después de completar el bucle y realizar el cálculo del promedio.

En el ejemplo incorrecto, el `return` está dentro del bucle debido a su indentación, lo que haría que la función retornara después de procesar solo el primer número. En el ejemplo correcto, el `return` está fuera del bucle, así que la función calcula el promedio de todos los números como se espera.

Este tipo de errores son comunes en programadores principiantes y pueden ser difíciles de detectar visualmente. Por fortuna, la mayoría de IDEs modernos incluyen herramientas para verificar la consistencia de la indentación.

Variables y tipos de datos

Almacenando y manipulando datos en tu código.



Variables en Python

En Python, una variable es un nombre que hace referencia a un valor almacenado en la memoria del ordenador. A diferencia de otros lenguajes, Python no requiere declarar explícitamente el tipo de una variable antes de usarla, ya que es un lenguaje de tipado dinámico.

Características de las variables en Python:

Creación y Asignación

- Se crean al asignarles un valor con el operador =
- No necesitan declaración previa de tipo
- Pueden cambiar de tipo durante la ejecución

Nomenclatura

- Deben comenzar con letra o guion bajo
- Pueden contener letras, números y guiones bajos
- Son sensibles a mayúsculas/minúsculas
- No pueden ser palabras reservadas

Convenciones

- snake_case para variables y funciones
- CamelCase para clases
- MAYÚSCULAS para constantes
- _nombre para variables "privadas"

Python utiliza un sistema de "referencias", lo que significa que las variables no almacenan directamente los valores, sino que apuntan a la ubicación en memoria donde se encuentra el valor. Esto tiene implicaciones importantes cuando se trabaja con tipos de datos mutables e inmutables.

Tipos de datos en Python

Python ofrece diversos tipos de variables para almacenar diferentes tipos de datos. Comprender estos tipos es fundamental para escribir código eficiente y evitar errores.

Tipos Numéricos

- int: Números enteros (5, -3, 42)
- float: Números con decimales (3.14, -0.001)
- complex: Números complejos (3+2j)

Tipos de Secuencia

- str: Cadenas de texto ('Hola', "Python")
- list: Listas mutables ([1, 2, 'tres'])
- tuple: Tuplas inmutables ((1, 2, 'tres'))

Tipos Booleanos y Nulos

- bool: Valores lógicos (True, False)
- None: Valor nulo o ausencia de valor

Tipos avanzados

- dict: Diccionarios {clave: valor}
- set: Conjuntos de elementos únicos
- frozenset: Conjuntos inmutables

Python también permite crear tipos personalizados mediante clases. El tipo de una variable puede verificarse en tiempo de ejecución utilizando la función `type()`, y se puede convertir entre tipos compatibles mediante funciones como `int()`, `float()`, `str()`, etc.

Tipos de Datos Numéricos en Python

Python maneja tres tipos numéricos principales para representar diferentes formas de números. Comprender sus características es clave para operaciones matemáticas precisas y eficientes.



`int` (Enteros)

- Definición: Números enteros, sin parte fraccionaria. Pueden ser positivos, negativos o cero.
- Precisión: Ofrecen precisión arbitraria, lo que significa que su tamaño solo está limitado por la memoria disponible del sistema.
- Uso: Ideal para contadores, índices, y cualquier cantidad que requiera un valor exacto sin decimales.
- Ejemplo: `edad = 30`, `poblacion_mundial = 8000000000`



`float` (Flotantes)

- Definición: Números reales que incluyen una parte decimal o un exponente.
- Precisión: Utilizan una representación de punto flotante de doble precisión (64 bits), lo que puede llevar a imprecisiones en cálculos muy específicos.
- Uso: Cálculos científicos, monetarios (con precaución), mediciones.
- Ejemplo: `precio = 19.99`, `pi = 3.14159`



`complex` (Complejos)

- Definición: Números con una parte real y una parte imaginaria, donde la parte imaginaria se denota con 'j' o 'J'.
- Características: La parte imaginaria siempre es un número flotante. Son inmutables.
- Uso: Aplicaciones en ingeniería eléctrica, procesamiento de señales y matemáticas avanzadas.
- Ejemplo: `z = 2 + 3j`, `frecuencia = 5.2 - 1.8j`

Los tres tipos numéricos son inmutables, lo que significa que una vez creados, su valor no puede ser modificado. Cualquier operación que parezca "modificar" un número en realidad crea un nuevo objeto numérico.

Tipos de Datos de Secuencia y Mapeo

Más allá de los números, Python ofrece tipos de datos potentes para organizar y manipular colecciones de información. Estos incluyen secuencias para datos ordenados y mapeos para datos estructurados con claves.

`str` (Cadenas de Texto)

Las cadenas son secuencias inmutables de caracteres Unicode. Se usan para representar texto.

- Inmutables: No se pueden modificar una vez creadas.
- Operaciones: Concatenación (+), repetición (*), segmentación (slicing), y métodos como `.upper()`, `.split()`.
- Ejemplo: `saludo = "Hola Mundo"`
- Se pueden escribir con comillas dobles o simples, pero hay que ser coherente con el uso.

`list` (Listas)

Las listas son colecciones ordenadas y mutables de elementos. Pueden contener datos de distintos tipos.

- Mutables: Sus elementos se pueden añadir, eliminar o modificar.
- Acceso: Por índice (`mi_lista[0]`).
- Ejemplo: `frutas = ["manzana", "banana", "uva"]`

`tuple` (Tuplas)

Las tuplas son colecciones ordenadas e inmutables de elementos, similares a las listas pero sin la posibilidad de modificación.

- Inmutables: Una vez creadas, no se pueden alterar.
- Uso: Cuando se necesita una secuencia fija de elementos o como claves de diccionario.
- Ejemplo: `punto = (10, 20)`

`dict` (Diccionarios)

Los diccionarios son colecciones no ordenadas de pares clave-valor, donde cada clave es única y apunta a un valor.

- Mutables: Se pueden añadir, eliminar y modificar pares clave-valor.
- Acceso: Por clave (`mi_dict["clave"]`).
- Ejemplo: `persona = {"nombre": "Ana", "edad": 30}`

Elegir el tipo de datos adecuado es crucial para la eficiencia y la legibilidad del código. Cada tipo tiene sus propias ventajas y casos de uso óptimos, dependiendo de si los datos deben ser mutables, ordenados o accesibles por una clave.

Tipos de Datos: Booleano y Nulo

Además de los tipos numéricos y de secuencia, Python ofrece tipos de datos específicos para la lógica de programación y para representar la ausencia de valor. Comprender `bool` y `None` es fundamental para el control de flujo y la gestión de estados en tu código.



`bool` (Booleano)

Definición: Representa valores de verdad, siendo los pilares de la lógica condicional en Python.

- Valores: Solo puede ser `True` (verdadero) o `False` (falso). Son sensibles a mayúsculas.
- Naturaleza: Es una subclase de `int`, donde `True` se evalúa como `1` y `False` como `0`.
- Uso: Fundamental en estructuras de control (`if`, `while`) y operaciones lógicas (`and`, `or`, `not`).
- Ejemplo: `tiene_permiso = True`, `es_mayor = (edad > 18)`



`None` (Nulo)

Definición: Es un objeto especial que representa la ausencia de un valor significativo o un valor nulo.

- Singleton: Solo existe una instancia de `None` en Python (`NoneType`).
- Uso: Comúnmente utilizado como un marcador de posición para variables que aún no tienen un valor, o como valor de retorno implícito para funciones que no devuelven nada explícitamente.
- Operaciones: No admite operaciones aritméticas ni la mayoría de los métodos de otros tipos de datos.
- Comparación: Se comprueba su existencia con el operador de identidad `is` (e.g., `if variable is None:`).
- Ejemplo: `valor_opcional = None`, `def funcion_sin_retorno(): pass`

Mientras que `bool` define el estado lógico, `None` se encarga de la representación de la indefinición o la ausencia, ambos vitales para la robustez y claridad del Código
Un singleton es un concepto que se refiere a que solo puede existir una única instancia de un objeto o valor en todo el programa.

Variables: Ejemplos Prácticos

```
# Variables numéricas
edad = 25                    # int
altura = 1.75                # float
numero_complejo = 3 + 2j     # complex

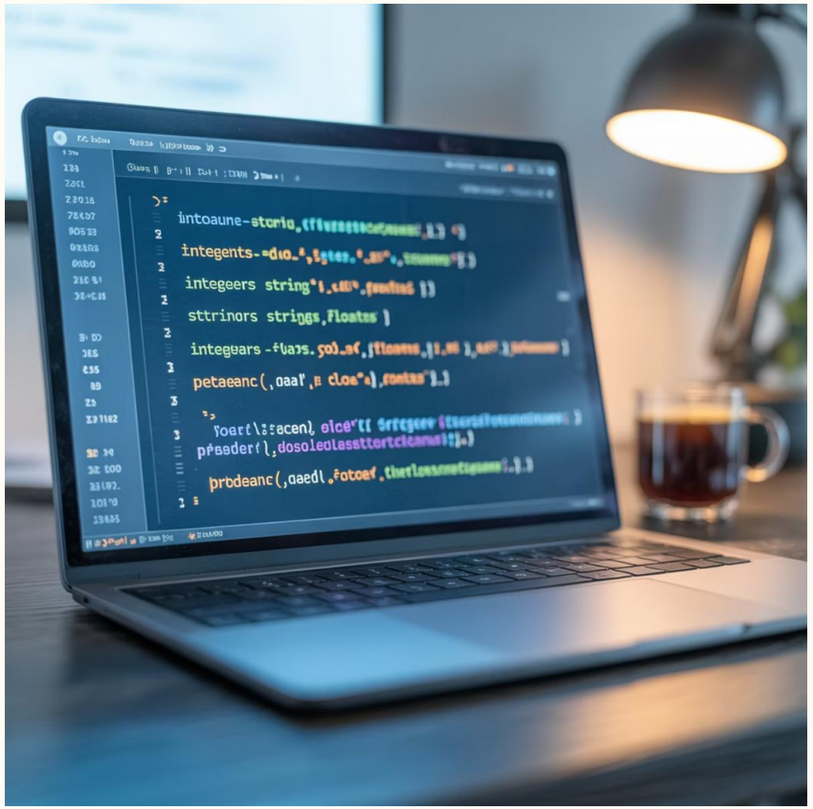
# Variables de texto
nombre = "Ana García"        # str
mensaje = 'Hola Mundo'       # str (comillas simples o dobles)

# Colecciones
notas = [7, 8, 9, 6, 10]     # list (mutable)
coordenadas = (40.4, -3.7) # tuple (inmutable)
usuario = {                  # dict
    "nombre": "Carlos",
    "edad": 30,
    "activo": True
}
colores = {"rojo", "verde", "azul"}  # set

# Booleanos
es_adulto = True             # bool
tiene_permiso = False        # bool

# Valor nulo
datos_pendientes = None      # NoneType

# Verificación de tipos
print(type(edad))            #
print(type(nombre))          #
print(type(notas))           #
```



El tipado dinámico de Python permite asignar diferentes tipos de valores a la misma variable en distintos momentos:

```
x = 10       # x es un int
x = "texto"  # ahora x es un str
x = [1, 2, 3] # ahora x es una list
```

Esta flexibilidad es muy útil, pero también puede llevar a errores sutiles si no se tiene cuidado con los cambios de tipo.

Utilidad Específica de Cada Tipo de Variable

Tipos Numéricos

- int: Contadores, índices, operaciones matemáticas exactas
- float: Mediciones, cálculos científicos, porcentajes
- complex: Cálculos de ingeniería, procesamiento de señales

Cadenas (str)

- Manejo de texto, nombres, descripciones
- Procesamiento de datos textuales
- Generación de informes y mensajes
- Manipulación de archivos (rutas, nombres)

Listas (list)

- Colecciones ordenadas y modificables
- Almacenamiento de datos heterogéneos
- Implementación de pilas y colas
- Manipulación dinámica de colecciones

Diccionarios (dict)

- Mapeo de claves a valores
- Acceso rápido por clave (no por posición)
- Representación de estructuras de datos complejas
- Configuraciones y parámetros

Conjuntos (set)

- Eliminación de duplicados
- Operaciones matemáticas de conjuntos
- Verificación rápida de pertenencia

Tuplas (tuple)

- Datos que no deben modificarse
- Claves de diccionarios (inmutables)
- Retorno múltiple de funciones
- Más eficientes que las listas

La elección del tipo de variable adecuado es crucial para el rendimiento, la claridad y la corrección de vuestro código. Por ejemplo, si necesitáis una colección donde no importen los duplicados ni el orden, un conjunto (set) será mucho más eficiente que una lista.

Mutabilidad e Inmutabilidad

Un concepto fundamental para entender cómo funcionan las variables en Python es la distinción entre tipos mutables e inmutables. Esta característica determina si un objeto puede ser modificado después de su creación.

Tipos Inmutables

- int, float, complex
- str
- tuple
- frozenset
- bytes

Una vez creados, su contenido no puede modificarse. Cualquier operación que parezca modificarlos en realidad crea un nuevo objeto.

Tipos Mutables

- list
- dict
- set
- bytearray
- Las instancias de clases personalizadas (por defecto)

Su contenido puede ser modificado después de la creación, manteniendo la misma identidad del objeto.

Entender la mutabilidad es crucial porque afecta a:

- El comportamiento al pasar variables a funciones
- La creación de copias de objetos
- El uso de objetos como claves de diccionarios
- El rendimiento de ciertas operaciones

Por ejemplo, solo los tipos inmutables pueden usarse como claves de diccionarios, ya que las claves no deben cambiar durante la vida del diccionario.

Ejemplo: Mutabilidad vs Inmutabilidad

```
# Ejemplo con tipo inmutable (str)
nombre = "Juan"
print(id(nombre))  # Imprime la identidad (dirección de memoria)

nombre = nombre + " Pérez"  # Parece modificar, pero crea un nuevo objeto
print(id(nombre))  # ¡La identidad ha cambiado! Es un nuevo objeto

# Ejemplo con tipo mutable (list)
edades = [25, 30, 35]
print(id(edades))  # Imprime la identidad

edades.append(40)  # Modifica la lista original
print(id(edades))  # La identidad NO ha cambiado, es el mismo objeto
```

En este ejemplo se muestra claramente la diferencia entre tipos mutables e inmutables. La cadena (str) no se modifica realmente, sino que cada operación crea una nueva. La lista (list), en cambio, sí se modifica directamente. Esto tiene importantes implicaciones cuando pasamos parámetros a funciones: los objetos mutables pasados como argumentos pueden ser modificados dentro de la función, afectando al objeto original fuera de la función.

Constantes en Python

A diferencia de otros lenguajes de programación, Python no tiene un mecanismo integrado para declarar constantes verdaderas (valores que no pueden modificarse). Sin embargo, existe una convención ampliamente aceptada para indicar que una variable debe tratarse como una constante.

Convención para constantes:

Las constantes se definen utilizando letras mayúsculas y guiones bajos para separar palabras:

```
PI = 3.14159
GRAVEDAD = 9.8
MAX_USUARIOS = 100
COLOR_PRINCIPAL = "#325F7B"
DIAS_SEMANA = ("Lunes", "Martes", "Miércoles",
               "Jueves", "Viernes", "Sábado",
               "Domingo")
```

Esta es solo una convención, no una restricción técnica. Técnicamente, estas "constantes" pueden modificarse, pero hacerlo va en contra de las buenas prácticas de programación.

Buenas prácticas:

- Definir las constantes al principio del módulo
- Usar nombres descriptivos
- Documentar su propósito con comentarios
- Para proyectos grandes, crear un módulo específico de constantes

```
# constants.py
DATABASE_HOST = "localhost"
DATABASE_PORT = 5432
API_KEY = "abc123xyz"
DEBUG_MODE = True
```

Y luego importarlas donde sea necesario:

```
from constants import API_KEY, DEBUG_MODE
```

Literales en Python

Los literales son representaciones de valores fijos en el código. Son la forma más básica de expresar un valor en Python, y pueden ser de diferentes tipos:

```
print(42)           # 42 es un literal entero
print(3.14)         # 3.14 es un literal decimal (float)
```

```
print("Hola")       # "Hola" es un literal de texto
print(True)         # True es un literal booleano
```

Literales Numéricos

- Enteros: 42, -7, 0
- Enteros con base explícita: 0b101 (binario), 0o17 (octal), 0xA3 (hexadecimal)
- Flotantes: 3.14, -0.001, 2e10 (notación científica)
- Complejos: 3+4j, -1.5+2.2j

Literales de Texto

- Cadenas: 'hola', "Python", """Texto con múltiples líneas"""
- Cadenas formateadas: f"El valor es {x}"
- Cadenas raw: r"\n no es salto de línea"
- Caracteres: No existe un tipo específico, se usa str de longitud 1

Literales Booleanos y Nulos

- Booleanos: True, False
- Nulo: None

Literales de Colección

- Listas: [1, 2, 3], ['a', 'b', 'c']
- Tuplas: (1, 2, 3), ('a', 'b', 'c')
- Diccionarios: {"a": 1, "b": 2}
- Conjuntos: {1, 2, 3}

Los literales son la forma más directa de incluir datos en vuestro código. A diferencia de las variables, no pueden ser modificados durante la ejecución del programa, ya que representan valores específicos e inmutables.

Ejemplos de Uso de Literales

```
# Literales numéricos
precio = 19.99  # Literal float
descuento = 0.15  # Literal float
cantidad = 5  # Literal int

# Cálculo usando literales y variables
total = precio * cantidad * (1 - descuento)
print(f"Total a pagar: {total}€")

# Literales de texto
mensaje = "Bienvenido a la tienda"
error = 'Producto no disponible'

# Literal multilinea
descripcion = """Este producto tiene
múltiples características:
- Alta calidad
- Bajo consumo
- Garantía extendida"""

# Literales de colección
colores_disponibles = ["rojo", "verde", "azul"]
tallas = ("S", "M", "L", "XL")
producto = {
    "nombre": "Camiseta",
    "precio": 19.99,
    "disponible": True
}
```

Diferencia entre Variables, Constantes y Literales

Literales

Son valores fijos escritos directamente en el código fuente.

- No tienen nombre
- No ocupan una ubicación con nombre en memoria
- Ejemplos: 42, "Hola", True

Variables

Son nombres que se refieren a valores que pueden cambiar durante la ejecución del programa.

- Se crean mediante asignación
- Su valor puede modificarse
- Ejemplo: contador = 0

Constantes

Son variables cuyo valor no debería cambiar una vez definido (por convención en Python).

- Se escriben en MAYÚSCULAS
- Se definen una sola vez
- Ejemplo: PI = 3.14159

Comprender estas diferencias es fundamental para escribir código claro y mantenible:

- Los literales son los valores más básicos en Python, y se utilizan para inicializar variables y constantes.
- Las variables son recipientes para datos que necesitan ser manipulados o que pueden cambiar durante la ejecución.
- Las constantes (por convención) representan valores fijos que dan significado al código y hacen que sea más mantenible y legible.

Operadores en Python



Operadores en Python

Los operadores son símbolos especiales que realizan operaciones sobre variables y valores. Python proporciona varios tipos de operadores para realizar diferentes tipos de operaciones.

Principales tipos de operadores:

Operadores Aritméticos

- +: Suma
- -: Resta
- *: Multiplicación
- /: División (resultado float)
- //: División entera
- %: Módulo (resto)
- **: Potencia

Operadores de Comparación

- ==: Igual a
- !=: No igual a
- >: Mayor que
- <: Menor que
- >=: Mayor o igual que
- <=: Menor o igual que

Operadores Lógicos

- and: Y lógico
- or: O lógico
- not: Negación

Operadores de Identidad

- is: Identidad
- is not: No identidad

Operadores de Pertenencia

- in: Pertenencia
- not in: No pertenencia

Operadores de Asignación

- =: Asignación simple
- +=, -=, *=, /=: Asignación con operación.

Operadores Aritméticos: Ejemplos

```
# Ejemplos de operadores aritméticos
a = 10
b = 3

# Operaciones básicas
suma = a + b         # 13
resta = a - b        # 7
multiplicacion = a * b  # 30
division = a / b     # 3.3333... (resultado siempre float)
division_entera = a // b  # 3 (parte entera)
modulo = a % b       # 1 (resto de la división)
potencia = a ** b    # 1000 (10 elevado a 3)

# Operadores con diferentes tipos de datos
x = 5
y = 2.5
z = x + y  # 7.5 (int + float = float)

# Operadores con cadenas
nombre = "Juan"
apellido = "Pérez"
nombre_completo = nombre + " " + apellido  # "Juan Pérez"
repetido = nombre * 3  # "JuanJuanJuan"
```



Consideraciones importantes:

- La división (/) siempre devuelve un float, incluso si el resultado es un número entero.
- La división entera (//) devuelve un int si los operandos son int, o float si alguno es float.
- Cuando mezclamos tipos numéricos, Python convierte automáticamente al tipo más complejo (ej: int → float).
- El operador + también sirve para concatenar cadenas y listas.
- El operador * con cadenas y listas realiza repetición, no multiplicación matemática.

Operadores de Comparación y Lógicos: Ejemplos

Operadores de Comparación

```
a = 5
b = 10
c = 5

# Comparaciones básicas
igual = a == c        # True
no_igual = a != b     # True
mayor = b > a         # True
menor = a < b         # True
mayor_igual = a >= c  # True
menor_igual = a <= c  # True

# Encadenamiento de comparaciones
en_rango = 1 <= a <= 10  # True
# Equivalente a: 1 <= a and a <= 10

# Comparación de strings (comparación lexicográfica)
print("abc" < "def")  # True
print("abc" < "ABC")  # False (mayúsculas vienen antes)

# Comparación de distintos tipos
print(1 == 1.0)  # True (valor equivalente)
print(1 is 1.0)  # False (objetos diferentes)
```

Operadores Lógicos

```
x = True
y = False

# Operaciones lógicas básicas
and_result = x and y # False
or_result = x or y # True
not_result = not x # False

# Combinando operadores
resultado = (a < b) and (b < 15) # True

# Cortocircuito en operadores lógicos
result1 = False and print("No se ejecuta")
# No se imprime nada debido al cortocircuito

result2 = True or print("No se ejecuta")
# No se imprime nada debido al cortocircuito

# Operadores de identidad
lista1 = [1, 2, 3]
lista2 = [1, 2, 3]
lista3 = lista1

print(lista1 == lista2) # True (mismo valor)
print(lista1 is lista2) # False (diferente objeto)
print(lista1 is lista3) # True (mismo objeto)
```

Otros Operadores: Ejemplos

Operadores de Pertenencia

```
# Comprobación de pertenencia en listas
frutas = ["manzana", "naranja", "plátano"]
tiene_manzana = "manzana" in frutas     # True
no_tiene_uva = "uva" not in frutas      # True

# Comprobación en strings
texto = "Python es fantástico"
contiene_python = "Python" in texto     # True
contiene_java = "Java" in texto         # False

# Comprobación en diccionarios (busca en claves)
usuario = {"nombre": "Ana", "edad": 28}
tiene_nombre = "nombre" in usuario      # True
tiene_direccion = "dirección" in usuario # False

# Comprobación en conjuntos
numeros = {1, 2, 3, 4, 5}
tiene_tres = 3 in numeros               # True
tiene_seis = 6 in numeros               # False
```

Operadores de Asignación

```
# Asignación simple
x = 10

# Asignación compuesta
x += 5 # Equivale a: x = x + 5
print(x) # 15

x -= 3 # Equivale a: x = x - 3
print(x) # 12

x *= 2 # Equivale a: x = x * 2
print(x) # 24

x /= 4 # Equivale a: x = x / 4
print(x) # 6.0 (ahora es float)

x //= 2 # Equivale a: x = x // 2
print(x) # 3.0 (sigue siendo float)

x **= 3 # Equivale a: x = x ** 3
print(x) # 27.0

# Asignación con strings
s = "Hola"
s += " Mundo" # "Hola Mundo"

# Asignación múltiple
a, b, c = 1, 2, 3

# Intercambio de valores
a, b = b, a # a=2, b=1
```

Operadores Bitwise

Los operadores bitwise (a nivel de bits) trabajan con los números como secuencias de bits (0s y 1s). Son menos utilizados en programación general, pero muy importantes en ciertos contextos como programación de sistemas, procesamiento de datos binarios o desarrollo de software de bajo nivel.

Operadores Bitwise Básicos

- &: AND a nivel de bits
- |: OR a nivel de bits
- ^: XOR a nivel de bits
- ~: NOT a nivel de bits (complemento a uno)

Operadores de Desplazamiento

- <<: Desplazamiento a la izquierda
- >>: Desplazamiento a la derecha

```
# Ejemplos de operadores bitwise
a = 60  # 0011 1100 en binario
b = 13  # 0000 1101 en binario

# Operaciones bitwise
c = a & b   # 0000 1100 → 12 (AND)
d = a | b   # 0011 1101 → 61 (OR)
e = a ^ b   # 0011 0001 → 49 (XOR)
f = ~a      # 1100 0011 → -61 (NOT)

# Desplazamientos
g = a << 2  # 1111 0000 → 240 (desplaz. izquierda)
h = a >> 2  # 0000 1111 → 15 (desplaz. derecha)
```



Aplicaciones prácticas:

- Manipulación de banderas y estados
- Operaciones con máscaras de bits
- Algoritmos de encriptación
- Optimización de memoria en sistemas embebidos
- Procesamiento de imágenes a bajo nivel
- Manipulación de protocolos de red

El desplazamiento a la izquierda (<<) es equivalente a multiplicar por 2^n, mientras que el desplazamiento a la derecha (>>) es equivalente a dividir por 2^n (donde n es el número de posiciones desplazadas). Esto hace que sean operaciones muy eficientes para ciertas multiplicaciones y divisiones.

Precedencia de Operadores

Cuando una expresión contiene múltiples operadores, la precedencia de operadores determina el orden en que se evalúan las operaciones. Los operadores con mayor precedencia se evalúan antes que los de menor precedencia.

Orden de precedencia (de mayor a menor):

1. Paréntesis: (expresión)
2. Potencia: **
3. Operadores unarios: +x, -x, ~x
4. Multiplicación, división: *, /, //, %
5. Suma, resta: +, -
6. Desplazamientos: <<, >>
7. AND a nivel de bits: &
8. XOR a nivel de bits: ^
9. OR a nivel de bits: |
10. Comparaciones: ==, !=, >, >=, <, <=, is, is not, in, not in
11. NOT lógico: not
12. AND lógico: and
13. OR lógico: or

Los operadores con la misma precedencia se evalúan de izquierda a derecha, excepto la potencia (**) que se evalúa de derecha a izquierda.

Recomendación: Cuando tengáis dudas sobre la precedencia, usad paréntesis para hacer el código más explícito y evitar errores. Además de garantizar el orden correcto, los paréntesis hacen el código más legible.

```
# Ejemplos de precedencia
resultado1 = 2 + 3 * 4       # 14, no 20
resultado2 = (2 + 3) * 4     # 20, no 14

# Operadores unarios tienen precedencia sobre
potencia
resultado3 = -2**2           # -4, no 4
resultado4 = (-2)**2         # 4, no -4

# Operadores lógicos
resultado5 = True or False and False
# True (and tiene mayor precedencia)
# Equivale a: True or (False and False)

# Encadenamiento de comparaciones
resultado6 = 1 < 2 < 3       # True
# Equivale a: 1 < 2 and 2 < 3

# Expresión compleja
resultado7 = 2 * 3**2 + (4 + 5) // 3
# 2 * 9 + 9 // 3
# 18 + 3
# 21
```

Clasificación de Operadores

Aritméticos

Realizan operaciones matemáticas básicas.

Ejemplos: +, -, *, /, %, //, **

Comparación

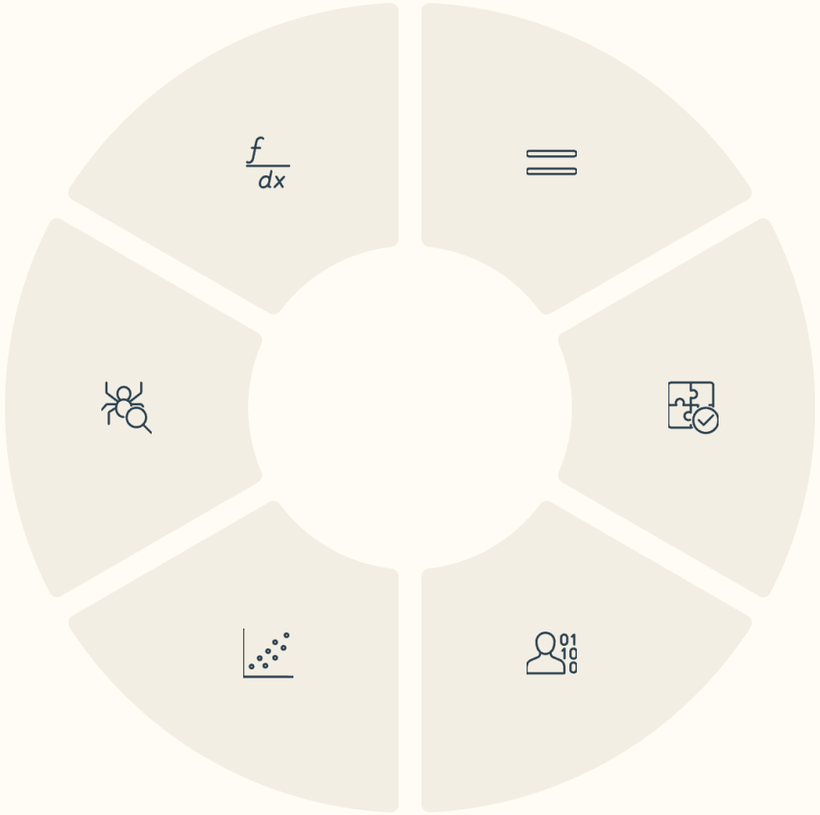
Comparan valores y devuelven booleanos.

Ejemplos: ==, !=, <, >, <=, >=

Pertenencia e Identidad

Verifican relaciones entre objetos.

Ejemplos: in, not in, is, is not



Lógicos

Combinan expresiones booleanas.

Ejemplos: and, or, not

Asignación

Asignan valores a variables.

Ejemplos: =, +=, -=, *=, /=

Bitwise

Operan a nivel de bits individuales.

Ejemplos: &, |, ^, ~, <<, >>

Esta clasificación de operadores es útil para entender mejor cómo se utilizan en diferentes contextos. Algunos operadores pueden pertenecer a más de una categoría según cómo se utilicen. Por ejemplo, el símbolo + es un operador aritmético cuando suma números, pero también se usa para concatenar cadenas o listas.

Cada tipo de operador tiene propósitos específicos y es importante conocer bien su funcionamiento para escribir código eficiente y libre de errores. La combinación adecuada de operadores os permitirá expresar algoritmos complejos de manera concisa y elegante.

Operadores en Expresiones

Una expresión en programación es una combinación de valores, variables, operadores y llamadas a funciones que se evalúa para producir un resultado. Python permite construir expresiones complejas combinando diferentes tipos de operadores.

Expresiones Aritméticas

```
# Expresión para calcular el área de un círculo
radio = 5
pi = 3.14159
area = pi * radio**2
print(f"Área: {area}")  # 78.53975

# Expresión para convertir temperatura
celsius = 25
fahrenheit = (celsius * 9/5) + 32
print(f"{celsius}°C = {fahrenheit}°F")  # 77°F

# Expresión para calcular el precio final
precio_base = 100
impuesto = 0.21
descuento = 0.05
precio_final = precio_base * (1 + impuesto) * (1 - descuento)
print(f"Precio final: {precio_final}€")  # 114.95€
```

Expresiones Lógicas y de Comparación

```
# Verificar si un número es par y positivo
numero = 24
es_par_positivo = numero > 0 and numero % 2 == 0
print(f"¿Es par y positivo? {es_par_positivo}")  # True

# Comprobar si un valor está en un rango
edad = 25
es_adulto_joven = 18 <= edad < 30
print(f"¿Es adulto joven? {es_adulto_joven}")  # True

# Evaluar múltiples condiciones
temperatura = 28
humedad = 85
lluvia = False

hace_calor = temperatura > 25
esta_humedo = humedad > 80
dia_agradable = hace_calor and esta_humedo and not lluvia
print(f"¿Es un día agradable? {dia_agradable}")  # True
```

Las expresiones pueden ser tan simples como una sola variable o tan complejas como sea necesario. La legibilidad es clave: si una expresión es demasiado compleja, considerad dividirla en partes más pequeñas o usar variables intermedias para hacerla más comprensible.

Conversiones de Tipos en Python

En Python, a menudo necesitaréis convertir datos de un tipo a otro para realizar operaciones específicas o asegurar la compatibilidad. Esta sección explora las diferentes formas de llevar a cabo estas conversiones, tanto de forma explícita como implícita.

Comprender cómo y cuándo convertir tipos es fundamental para evitar errores y escribir código robusto que maneje distintos formatos de datos de manera eficiente.



Conversión de Tipos en Python

La conversión de tipos (type casting) es el proceso de cambiar un valor de un tipo de datos a otro. Python proporciona dos formas de realizar estas conversiones:

Conversión Implícita

Python realiza automáticamente algunas conversiones de tipos cuando:

- Se realizan operaciones entre diferentes tipos numéricos
- Se evalúan expresiones booleanas

```
# Conversiones numéricas implícitas
entero = 10
flotante = 3.5
resultado = entero + flotante  # int + float → float
print(type(resultado))  #

# Conversiones implícitas a booleano
if 42:  # Número distinto de 0 → True
    print("Los números distintos de 0 son True")

if "Hola":  # String no vacío → True
    print("Las cadenas no vacías son True")

if []:  # Lista vacía → False
    print("Esto no se imprime")
```

Conversión Explícita

Para cambiar explícitamente el tipo de un valor, Python proporciona funciones de conversión:

```
# Conversiones numéricas explícitas
a = int(3.9) # 3 (trunca, no redondea)
b = float(5) # 5.0
c = complex(2) # 2+0j

# Conversiones a/desde string
d = int("42") # 42
e = float("3.14") # 3.14
f = str(123) # "123"
g = str(3.14159) # "3.14159"

# Conversiones a booleano
h = bool(0) # False
i = bool(42) # True
j = bool("") # False
k = bool("Hola") # True

# Conversiones a/desde listas, tuplas, etc.
l = list("Python") # ['P', 'y', 't', 'h', 'o', 'n']
m = tuple([1, 2, 3]) # (1, 2, 3)
n = set([1, 2, 2, 3]) # {1, 2, 3} (elimina duplicados)
o = dict([(1, "a"), (2, "b")]) # {1: 'a', 2: 'b'}

print(a, b, c)
print(d, e, f, g)
print(h, i, j, k)
print(l, m, n, o)
```

Errores Comunes en Conversión de Tipos

Errores en Conversiones Numéricas

```
# Intentar convertir strings no numéricos a números
try:
    numero = int("abc")
    # Error: invalid literal for int() with base 10: 'abc'

# Intentar convertir strings con formato incorrecto
try:
    numero = int("42.5")  # El punto decimal no es válido para
int()
    # Error: invalid literal for int() with base 10: '42.5'

# La forma correcta sería:
numero = int(float("42.5"))  # Primero a float, luego a int
print(numero)  # 42

# Pérdida de precisión al convertir float a int
x = int(9.9999)  # Trunca a 9, no redondea
print(x)  # 9
```

Errores en Otras Conversiones

```
# Convertir objetos no iterables a listas
try:
    lista = list(123)
    # Error: 'int' object is not iterable

# Convertir a diccionario con formato incorrecto
try:
    d = dict([(1, 2, 3), (4, 5)])  # Las tuplas deben tener 2 elementos
    # Error: dictionary update sequence element #0
    # has length 3; 2 is required

# Intentar convertir tipos incompatibles
try:
    complejo_a_int = int(3+4j)
    # Error: can't convert complex to int

# Conversiones que pueden ser engañosas
print(bool([0]))  # True (lista no vacía, aunque contenga solo 0)
print(bool(" "))  # True (string no vacío, aunque sea solo un espacio)
```

Para evitar estos errores, es recomendable verificar el tipo y formato de los datos antes de intentar convertirlos, especialmente cuando se trabaja con datos proporcionados por usuarios. La funcione `type()` pueden ser útiles para estas verificaciones.

También es importante tener en cuenta que algunas conversiones pueden provocar pérdida de información (como float a int) o cambios inesperados en el valor, por lo que debéis ser conscientes de las implicaciones de cada conversión.

Conversión de Tipos para Operaciones

En muchas situaciones, necesitamos convertir tipos para poder realizar operaciones entre valores de diferentes tipos o para preparar los datos para funciones específicas.

Casos de uso comunes:

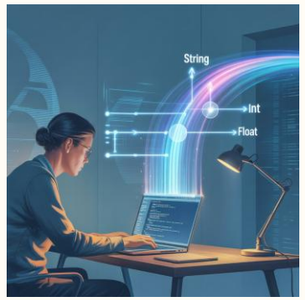
```
# Entrada de usuario (siempre viene como string)
entrada = input("Ingrese un número: ")  # "42"
numero = int(entrada)  # Convierte a entero para operar
resultado = numero * 2
print(f"El doble es: {resultado}")

# Formateo de salida
pi = 3.14159265359
pi_formateado = f"{pi:.2f}"  # Convierte a string con 2 decimales
print(f"Valor de π aproximado: {pi_formateado}")  # 3.14

# Combinación de strings y números
edad = 25
mensaje = "Tengo " + str(edad) + " años"
print(mensaje)  # "Tengo 25 años"

# Alternativa más eficiente con f-strings:
mensaje = f"Tengo {edad} años"
print(mensaje)  # "Tengo 25 años"

# Convertir entre colecciones
numeros = [1, 2, 3, 2, 1, 4]
numeros_unicos = list(set(numeros))  # Elimina duplicados
print(numeros_unicos)  # [1, 2, 3, 4]
```



Conversiones para JSON:

```
import json

# Datos en Python
datos_python = {
    "nombre": "Ana",
    "edad": 28,
    "activo": True,
    "hobbies": ["música", "deporte", "lectura"]
}

# Convertir Python a JSON string
json_str = json.dumps(datos_python)
print(json_str)
# {"nombre": "Ana", "edad": 28, "activo": true,
#  "hobbies": ["m\u00fasica", "deporte", "lectura"]}

# Convertir JSON string a Python
datos_recibidos = '{"id": 123, "estado": "completado"}'
objeto_python = json.loads(datos_recibidos)
print(objeto_python["estado"])  # "completado"
```

Las conversiones entre Python y JSON son muy comunes en el desarrollo de APIs y aplicaciones web.

Conversiones Implícitas: En Profundidad

Las conversiones implícitas ocurren automáticamente cuando Python necesita conciliar diferentes tipos de datos en una operación. Es importante entender cuándo y cómo ocurren estas conversiones para evitar comportamientos inesperados.

Reglas de conversión implícita:

Operaciones Numéricas

- int + int → int
- int + float → float
- float + float → float
- int/float + complex → complex

La conversión siempre va al tipo "más amplio" que puede representar ambos valores correctamente.

Contextos Booleanos

Python convierte implícitamente a bool en contextos que requieren un valor booleano:

- Números: 0 es False, cualquier otro número es True
- Strings: "" es False, cualquier otro string es True
- Colecciones: Vacías son False, no vacías son True
- None es siempre False




```
# Ejemplos de conversiones implícitas

# Operaciones numéricas
a = 10 # int
b = 3.5 # float
c = a + b # float (10.0 + 3.5)
print(f"Tipo de c: {type(c)}")

# División siempre produce float en Python 3
d = 10 / 5 # Resultado: 2.0, no 2
print(f"d = {d}, tipo: {type(d)}")

# Operador * con diferentes tipos
repetir = "abc" * 3 # string * int → string repetido
print(repetir) # "abcabcabc"

# Contextos booleanos
if 42:
 print("Ejecutado - 42 es True implícitamente")

contador = 10
while contador: # contador != 0 implícitamente
 contador -= 1
 if contador == 5:
 break

# Operador ternario usa conversión implícita
mensaje = "Aprobado" if 75 > 70 else "Suspendido"
print(mensaje) # "Aprobado"
```

Conversiones Explícitas: En Profundidad

Las conversiones explícitas (también llamadas type casting) os permiten controlar exactamente cuándo y cómo se cambia un tipo de dato a otro. Python proporciona funciones integradas para realizar estas conversiones.

Funciones de conversión principales:

- int(x, base=10): Convierte x a un entero. Opcionalmente, especifica la base para strings (2 para binario, 16 para hexadecimal, etc.)
- float(x): Convierte x a un número de punto flotante
- complex(real, imag=0): Crea un número complejo
- str(x): Convierte x a una cadena de texto
- bool(x): Convierte x a un valor booleano
- list(x), tuple(x), set(x): Convierten a estos tipos de colección
- dict(x): Convierte a diccionario (x debe tener formato adecuado)

```
# Conversiones numéricas avanzadas
binario = int("1010", 2)    # 10 (base 2)
octal = int("17", 8)        # 15 (base 8)
hexadecimal = int("1A", 16) # 26 (base 16)
print(f"Binario: {binario}, Octal: {octal}, Hex: {hexadecimal}")

# Formateo avanzado con diferentes bases
print(f"42 en binario: {bin(42)}")     # 0b101010
print(f"42 en octal: {oct(42)}")       # 0o52
print(f"42 en hexadecimal: {hex(42)}") # 0x2a
```

```
# Conversiones para manipulación de datos
datos = "1,2,3,4,5"
lista_strings = datos.split(",")  # ["1","2","3","4","5"]
lista_numeros = [int(x) for x in lista_strings]  # [1,2,3,4,5]
print(lista_numeros)

# Convertir entre tipos de colecciones
tupla_coordenadas = (40.4168, -3.7038)
lista_coordenadas = list(tupla_coordenadas)
lista_coordenadas.append(0)  # Ahora podemos modificarla
nueva_tupla = tuple(lista_coordenadas)
print(nueva_tupla)  # (40.4168, -3.7038, 0)

# Conversión bidireccional entre caracteres y códigos ASCII
caracter = 'A'
codigo = ord(caracter)  # 65
nuevo_caracter = chr(codigo)  # 'A'
print(f"'{caracter}' → {codigo} → '{nuevo_caracter}'")

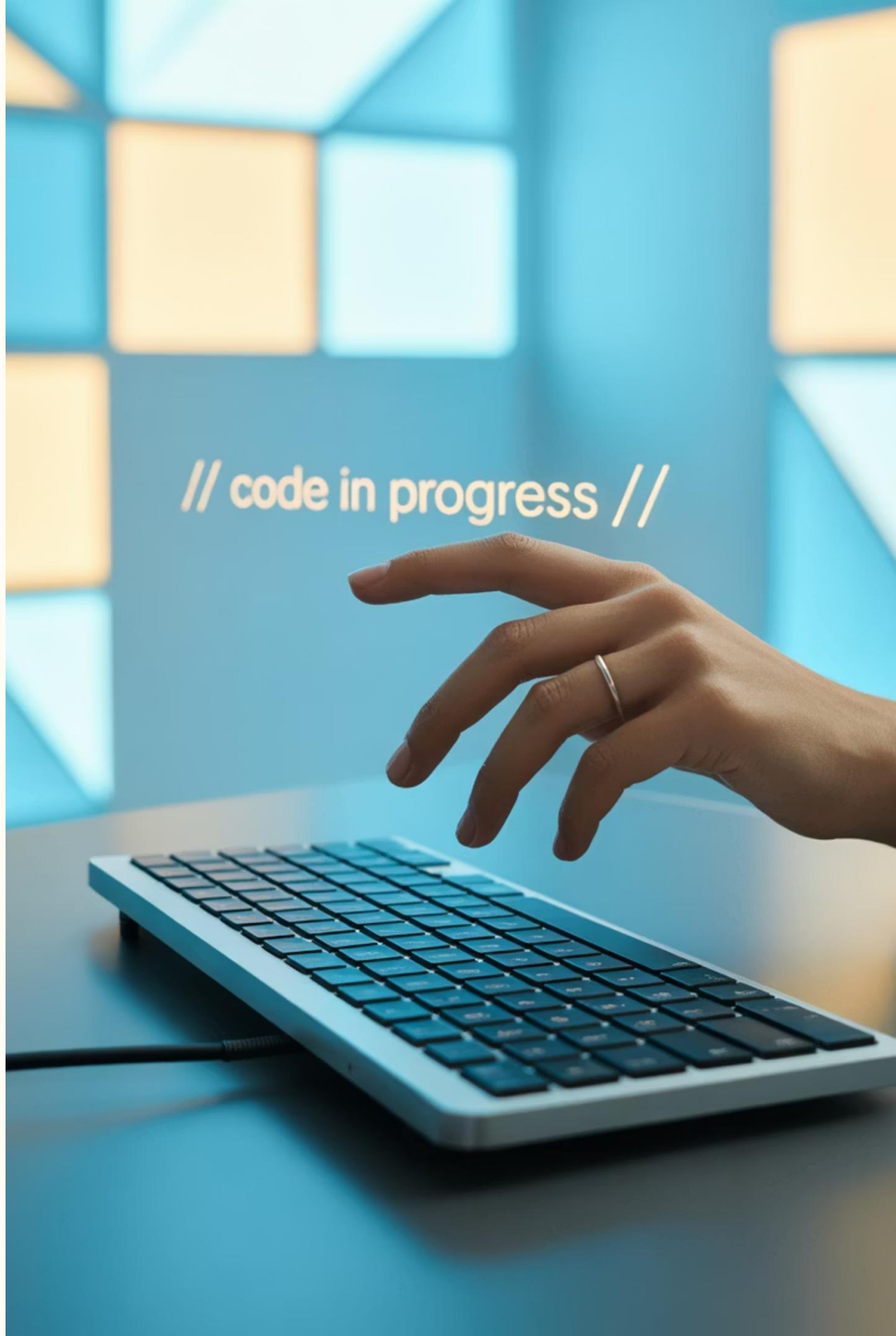
# Convertir entre tipos personalizados
class Punto:
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def __str__(self):
        return f"({self.x}, {self.y})"

punto = Punto(10, 20)
punto_str = str(punto)  # Usa el método __str__
print(punto_str)  # "(10, 20)"
```

La conversión explícita os da control total sobre cómo se transforman vuestros datos, lo que es crucial para garantizar la integridad y precisión de vuestras operaciones.

Comentarios en Python

Descubre cómo documentar tu código para hacerlo más legible y mantenible.



Comentarios en Python

Los comentarios son texto en el código fuente que no se ejecuta. Sirven para explicar el código, documentar decisiones de diseño, desactivar temporalmente código o facilitar el mantenimiento. Python ofrece dos tipos principales de comentarios.

Comentarios de una línea

Se crean utilizando el símbolo #. Todo lo que sigue a este símbolo hasta el final de la línea es ignorado por el intérprete de Python:

```
# Este es un comentario de una línea
x = 10  # Este comentario está al final de una instrucción

# Múltiples comentarios
# de una línea
# seguidos

"""
Aunque esto parece un comentario,
en realidad es una cadena de texto que no se asigna a ninguna variable
(pero Python lo evalúa como una expresión)
"""

# Desactivar código temporalmente (comentar código)
# resultado = funcion_que_falla()
resultado = valor_alternativo  # Usamos esto mientras tanto
```

Docstrings (cadenas de documentación)

Se crean utilizando tres comillas (simples o dobles). Se utilizan principalmente para documentar módulos, clases, funciones y métodos:

```
def calcular_area_rectangulo(largo, ancho):
    """
    Calcula el área de un rectángulo.

    Args:
        largo (float): Longitud del rectángulo
        ancho (float): Anchura del rectángulo

    Returns:
        float: El área calculada

    Examples:
        >>> calcular_area_rectangulo(5, 3)
        15.0
    """
    return largo * ancho

# Acceder a la documentación
print(calcular_area_rectangulo.__doc__)
help(calcular_area_rectangulo)
```

A diferencia de los comentarios con #, los docstrings son almacenados como atributos del objeto que documentan y pueden ser accedidos mediante el atributo __doc__ o la función help().

Buenas Prácticas para Comentarios

QUÉ comentar

- Explicar el "por qué", no el "qué"

  El código ya muestra qué hace; los comentarios deben explicar por qué se está haciendo algo, especialmente si no es obvio.

- Decisiones de diseño y algoritmos

  Explicar la lógica detrás de algoritmos complejos o decisiones de diseño importantes.

- Advertencias y consideraciones

  Indicar posibles problemas, efectos secundarios o consideraciones especiales.

- Referencias externas

  Enlaces a documentación, artículos o tickets relacionados con el código.

- TODOs y FIXMEs

  Marcar áreas que necesitan mejoras o correcciones futuras.

CÓMO comentar

Sea claro y conciso

Evita comentarios obvios o redundantes. Usa un lenguaje claro y directo.

Mantén los comentarios actualizados

Un comentario obsoleto es peor que ningún comentario, ya que puede llevar a confusiones.

Usa docstrings para documentación formal

Sigue las convenciones como NumPy, Google o reStructuredText para docstrings.

Escribe comentarios en inglés o en el idioma del equipo

Usa un idioma consistente en todo el proyecto, idealmente el mismo que las palabras clave del lenguaje.

Ejemplos de Comentarios en Código Real

Comentarios explicativos

```
def ordenar_por_eficiencia(algoritmos):
    """
    Ordena los algoritmos por eficiencia computacional.

    Los algoritmos se ordenan primero por complejidad temporal
    (O notation) y luego por uso de memoria.
    """
    # Usamos sorted con una clave personalizada en lugar de
    # sort() porque necesitamos preservar el original
    return sorted(algoritmos,
                  key=lambda x: (x.complejidad_temporal,
                                 x.uso_memoria))

# Este algoritmo usa el método de Newton-Raphson para
# encontrar la raíz cuadrada con alta precisión
def raiz_cuadrada(n, precision=0.0001):
    # Valor inicial: usamos n/2 como aproximación inicial
    # basado en experimentos que muestran convergencia más rápida
    x = n / 2

    while True:
        # Aplicamos la fórmula iterativa de Newton
        # x_next = x - f(x)/f'(x) donde f(x) = x² - n
        x_next = 0.5 * (x + n / x)

        # Si la diferencia es menor que la precisión requerida
        # hemos encontrado una aproximación suficientemente buena
        if abs(x - x_next) < precision:
            return x_next

        x = x_next
```

Comentarios de advertencia y TODOs

```
# ADVERTENCIA: Esta función modifica el diccionario original
# Considere hacer una copia si necesita preservar los datos
def normalizar_diccionario(dic):
 for clave in dic:
 if isinstance(dic[clave], str):
 dic[clave] = dic[clave].lower().strip()
 return dic

# TODO: Refactorizar esta función para mejorar rendimiento
# cuando trabaja con conjuntos de datos grandes (>1M elementos)
# Ver ticket #4567 para más detalles
def buscar_duplicados(lista):
 duplicados = []
 vistos = set()

 for elemento in lista:
 if elemento in vistos:
 duplicados.append(elemento)
 else:
 vistos.add(elemento)

 return duplicados

# FIXME: La función falla con matrices dispersas
# Implementación temporal hasta que se desarrolle una
# solución más robusta en la versión 2.0
def multiplicar_matrices(a, b):
 # Código de multiplicación...
```